

Locks

Basic Idea

- We have introduced how to protect a **critical section** using semaphores. Now we introduce a more basic way - lock
- Lock is type of variables
 - Each lock variable has
 - Two states: unlocked or locked
 - Unlocked: no thread holds the lock (also known as available or free)
 - Locked: exactly one thread holds the lock (known as acquired or held)

Pthread Locks

- The POSIX library provides mutual exclusion between threads
 - Also known as **mutex**

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

Or

```
int rc = pthread_mutex_init(&lock, NULL);
```

```
pthread_mutex_lock(&lock);
```

```
balance = balance + 1;
```

```
pthread_mutex_unlock(&lock);
```

Pthread Locks

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);  
int pthread_mutex_timedlock(pthread_mutex_t *mutex,  
                             struct timespec *abs_timeout);
```

- The `trylock()` returns failure if the lock is already held
- the `timedlock()` returns after a timeout or after acquiring the lock, whichever happens first

Building A Lock

- The perspective of a programmer
 - How should we build a lock?
 - What hardware support is needed?
 - What OS support is needed?
- Evaluating locks
 - Mutual exclusion
 - Fairness
 - Performance

Controlling Interrupts

- Disable interrupts for critical sections
 - One of the earliest solutions
 - Invented for single-processor systems
 - Very simple

```
1 void lock() {  
2     DisableInterrupts();  
3 }  
4 void unlock() {  
5     EnableInterrupts();  
6 }
```

```
1 void add() {  
2     lock();  
3     counter++;  
4     Unlock();  
5 }
```

Controlling Interrupts

- Cons
 - Allow any calling thread to perform a *privileged* operation (turning interrupts on and off), and thus *trust* that this facility is not abused
 - Does not work on multiprocessors
 - Turning off interrupts for extended periods of time can lead to interrupts becoming lost which can lead to serious systems problems
 - Inefficient
- Only used in kernel when OS accesses its own shared data structures for some cases

Test and Set

- **First Attempt: A Simple Flag**

```
typedef struct __lock_t { int flag; } lock_t;

void init (lock_t *mutex) {
    mutex->flag = 0; // 0 -> lock is available, 1 -> held
}

void lock (lock_t *mutex) {
    while (mutex->flag == 1) ; // spin-wait (do nothing)
    mutex->flag = 1; // now SET it!
}

void unlock(lock_t *mutex) { mutex->flag = 0; }
```


Two Problems

- Problem-1: Correctness

Thread 1 call lock()	Thread 2 call lock()
while(flag == 1) ;	
flag <- 0	
interrupt: switch to Thread 2	
	while(flag == 1);
	flag = 1;
	interrupt: switch to thread 1
flag = 1; // set flag to 1 (too!)	

Two Problems

- Problem-2: Performance
 - Spin-waiting: endlessly check the value of flag
 - Waste time waiting for another thread to release a lock
 - The waste is exceptionally high on a uniprocessor

Atomic Exchange

- Test-and-set instruction
 - Read and set memory atomically

Atomically perform

```
int TestAndSet(int *old_ptr, int new) {  
    int old = *old_ptr; // fetch old value at old_ptr  
    *old_ptr = new;      // store 'new' into old_ptr  
    return old;          // return the old value  
}
```

Spin Lock: Test-and-Set

```
typedef struct __lock_t { int flag; } lock_t;

void init (lock_t *lock) {
    lock->flag = 0; // 0 -> lock is available, 1 -> held
}

void lock (lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1)
        ; // spin-wait (do nothing)
}

void unlock(lock_t *lock) { lock->flag = 0; }
```

Evaluating Spin Locks

- Correctness
 - A correct lock
- Fairness
 - Spin locks don't provide any fairness guarantees
 - May lead to starvation
- Performance
 - Single CPU: quite painful
 - Multiple CPUs: reasonably well

Compare-And-Swap (x86)

Atomically perform

```
1 int CompareAndSwap(int *ptr, int expected, int new)
{
2     int actual = *ptr;
3     if (actual == expected)
4         *ptr = new;
5     return actual;
6 }
```

```
1 void lock(lock_t *lock) {
2     while (CompareAndSwap(&lock->flag, 0, 1) == 1)
3         ; // spin
4 }
```

Load-Linked and Store-Conditional (ARM)

```
1 int LoadLinked(int *ptr) {
2     return *ptr;
3 }
4
5 int StoreConditional(int *ptr, int value) {
6     if (no one has updated *ptr since the LoadLinked to this address) {
7         *ptr = value;
8         return 1; // success!
9     } else {
10        return 0; // failed to update
11    }
12 }
```

Load-Linked and Store-Conditional (ARM)

```
1 void lock(lock_t *lock) {
2     while (1) {
3         while (LoadLinked(&lock->flag) == 1)
4             ; // spin until it's zero
5         if (StoreConditional(&lock->flag, 1) == 1)
6             return; // if set-it-to-1 was a success: all done
7         // otherwise: try it all over again
8     }
9 }
10
11 void unlock(lock_t *lock) {
12     lock->flag = 0;
13 }
```


Ticket Lock (Fairness)

- Fetch-and-add instruction

Atomically perform

```
int FetchAndAdd(int *ptr) {  
    int old = *ptr; // fetch old value at ptr  
    *ptr = old + 1; // add 1 and store 'new' into ptr  
    return old;     // return the old value  
}
```

Ticket Lock: Fetch-and-Add

```
void init (lock_t *lock) {  
    lock->ticket = 0;  
    lock->turn   = 0;  
}
```

```
typedef struct __lock_t {  
    int ticket;  
    int turn;  
} lock_t;
```

```
void lock (lock_t *lock) {  
    int myturn = FetchAndAdd(&lock->ticket);  
    while (lock->turn != myturn) ← No starvation  
        ; // spin  
}
```

```
void unlock(lock_t *lock) { lock->turn = lock->turn + 1; }
```

How to Avoid Spinning (Performance)

- A Simple Approach: Just Yield
 - When you are going to spin, instead give up the CPU to another thread
 - OS primitive: `yield()`
 - State: Running → Ready
 - A plenty of waste from costly context switch and scheduling
 - Still has starvation

Lock with Test-and-Set and Yield

```
void init () { flag = 0; }
```

```
void lock () {  
    while (TestAndSet(&flag, 1) == 1)  
        yield(); // give up the CPU  
}
```

```
void unlock() { flag = 0; }
```

Is It Efficient?

- Two threads on one CPU
 - Yield() works quite well
 - A thread calls lock() and find a lock held
 - it will simply yield the CPU
 - the other thread will run and finish its critical section

Is It Efficient?

- 100 threads on one CPU
 - One thread acquires the lock and is preempted before releasing it
 - The other 99 will each call lock(), find the lock held, and yield the CPU
 - Assuming a round-robin scheduler, each of the 99 will execute this run-and-yield pattern
 - The cost of a context switch and sched can be substantial, and there is thus plenty of waste

Using Queue

- Previous problem: leave too much chance
 - Potentially lead to waste and starvation
- Explicitly select next thread to acquire lock
 - Queue: keep track of waiting threads
 - Support provided by Solaris
 - `Park()`: put a calling thread to sleep
 - `Unpark(threadID)`: wake a particular thread

Lock with Test-and-set, Yield and Wakeup

```
typedef struct __lock_t {  
    int flag;  
    int guard;  
    queue_t *q;  
} lock_t;
```

```
void lock_init (lock_t *lock) {  
    lock->flag = 0;  
    lock->guard = 0;  
    queue_init(lock->q);  
}
```


Lock with Test-and-set, Yield and Wakeup

```
void lock (lock_t *lock) {
    while (TestAndSet(&lock->guard, 1) == 1)
        ; // acquire guard lock by spinning
    if (lock->flag == 0) {
        lock->flag = 1; // lock is acquired
        lock->guard = 0;
    } else {
        queue_add(lock->q, gettid());
        lock->guard = 0;
        park();
    }
}
```

Lock with Test-and-set, Yield and Wakeup

```
void unlock (lock_t *lock) {
    while (TestAndSet(&lock->guard, 1) == 1)
        ; // acquire guard lock by spinning
    if (queue_empty(lock->q)) {
        // let go of lock; no one wants it
        lock->flag = 0;
    } else {
        // hold lock (for next thread!)
        unpark(queue_remove(lock->q));
    }
    lock->guard = 0;
}
```

Question 1

```
void lock (lock_t *lock) {  
    while (TestAndSet(&lock->guard, 1) == 1)  
        ; // acquire guard lock by spinning  
    if (lock->flag == 0) {  
        lock->flag = 1; // lock is acquired  
        lock->guard = 0;  
    } else {  
        queue_add(lock->q, gettid());  
        park();  
        lock->guard = 0;  
    }  
}
```

**What would happen if the release
of guard come after park() ?**

Deadlock: sleep with holding a spinlock

Question 2

```
void unlock (lock_t *lock) {  
    while (TestAndSet(&lock->guard, 1) == 1)  
        ; // acquire guard lock by spinning  
    if (queue_empty(lock->q)) {  
        // let go of lock; no one wants it  
        lock->flag = 0;  
    } else {  
        // hold lock (for next thread!)  
        unpark(queue_remove(lock->q));  
    }  
    lock->guard = 0;  
}
```

Why not reset flag (remain to 1)?

The waked thread will not set flag

Lock with Test-and-set, Yield and Wakeup

```
void lock (lock_t *lock) {
    while (TestAndSet(&lock->guard, 1) == 1)
        ; // acquire guard lock by spinning
    if (lock->flag == 0) {
        lock->flag = 1; // lock is acquired
        lock->guard = 0;
    } else {
        queue_add(lock->q, gettid());
        lock->guard = 0;
        park(); //missing the unpark notification?
    }
}
```

Wakeup/waiting Race

Thread 1 call lock() ... Thread 2 call unlock() ...

```
queue_add(Lock->q, mytid)
```

```
lock->guard = 0;
```

```
interrupt: switch to Thread 2
```

```
unpark();
```

```
interrupt: switch to Thread 1
```

```
park(); //mytid has been removed in unpark()
```

Wakeup/waiting Race

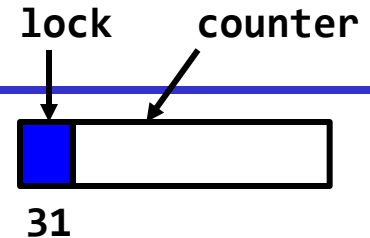
```
void lock (lock_t *lock) {
    while (TestAndSet(&lock->guard, 1) == 1)
        ; // acquire guard lock by spinning
    if (lock->flag == 0) {
        lock->flag = 1; // lock is acquired
        lock->guard = 0;
    } else {
        queue_add(lock->q, gettid());
        setpark();
        lock->guard = 0;
        park();
    }
}
```

Solaris:
setpark()
After calling it,
Park will return
immediately
instead of
sleeping if
another thread
just finished
unpark()

Different OS, Different Support

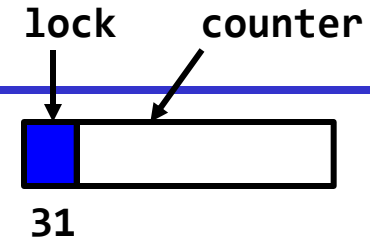
- Linux: `futex`
 - Each `futex` has associated with a specific physical memory local, and a per-`futex` in-kernel queue
 - `futex_wait(address, expected)`
 - If `*address == expected`, puts the calling thread to sleep
 - If not, the call return immediately
 - `futex_wake(address)`
 - Wake one thread that is waiting on the queue

Linux-based Futex Locks



```
void mutex_lock (int *mutex) {  
    int v;  
    if (atomic_bit_test_set(mutex, 31) == 0) return;  
    atomic_increment (mutex); // add counter  
    while (1) {  
        if (atomic_bit_test_set(mutex, 31) == 0) {  
            atomic_decrement(mutex);  
            return;  
        }  
        v = *mutex;  
        if (v >= 0) continue; // lock has released  
        futex_wait(mutex, v);  
    }  
}
```

Futex Locks



```
void mutex_unlock (int *mutex) {  
    /* Adding 0x80000000 to the counter results in 0 if and  
       only if there are not other interested threads */  
    if (atomic_add_zero(mutex, 0x80000000))  
        return;  
  
    /* There are other threads waiting for this mutex,  
       wake one of them up. */  
    futex_wake(mutex);  
}
```

Two-stage Locks

- Two-stage Lock (hybrid)

- 1st stage: a simple test-and-set and a potential spin
- 2nd stage: the caller is put to sleep

```
- while (1) {  
    if (atomic_bit_test_set(mutex, 31) == 0) {  
        atomic_decrement(mutex);  
        return;  
    }  
    v = *mutex;  
    if (v >= 0) continue; // lock has released  
    futex_wait(mutex, v);  
}
```

1st stage

2nd stage

Q: What are the advantages of two-phase

design?

Efficient in both low-contention and high-contention cases